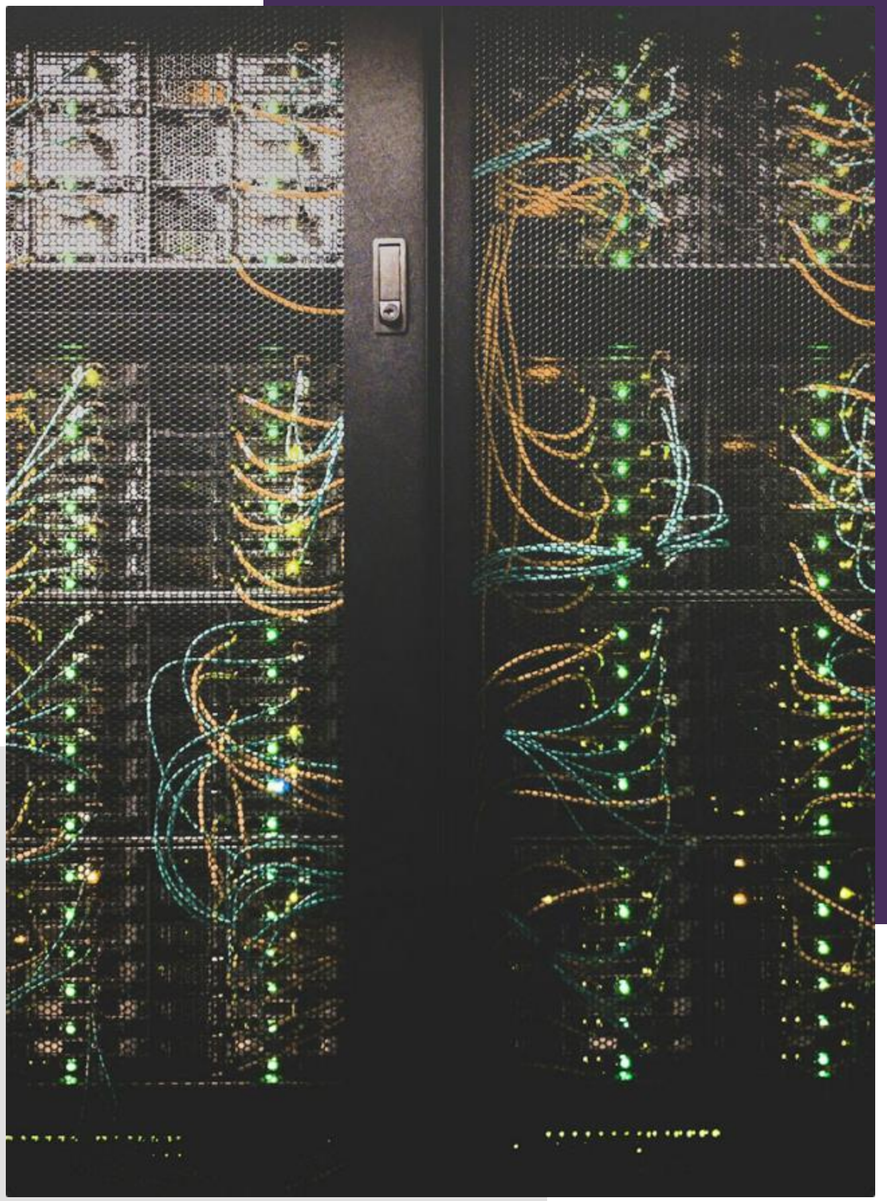


Automating the Machine Learning Lifecycle: A Modern MLOps Architecture

Building Robust, Scalable, and Self-Improving AI SystemsA deep dive into the technical architectures and automated engineering processes for AI.

Duangdao Wichadakul

Agenda

An overview of the continuous CI/CD/CT loops and technical deep dives into the MLOps workflow.

1 The MLOps Imperative

2 The Full Workflow

3 Stage 1: Data & Development

4 Stage 2: CI/CD & Testing

5 Stage 3: Registration & Deployment

6 Stage 4: Monitoring

7 Stage 5: Automated Retraining

8 Summary: Complete Toolchain

The MLOps Imperative

Comparing traditional research-heavy approaches with modern automated engineering principles.

Traditional "Research" Approach	Modern MLOps Approach
Process: Ad-hoc scripts in notebooks. Models are "thrown over the wall".	**Process:** A unified, automated pipeline from code commit to production.
Reproducibility: Extremely difficult. Relies on manual documentation.	**Reproducibility:** Guaranteed. Components versioned: code (Git), data (DVC), models.
Scalability: Very low. Each new model or update is a massive manual effort.	**Scalability:** High. Automated pipeline handles hundreds of models and updates.
Maintenance: Static. Models are left to decay in performance (model drift).	**Maintenance:** Dynamic. Continuous monitoring triggers automated retraining.

Stage 1: Data & Development

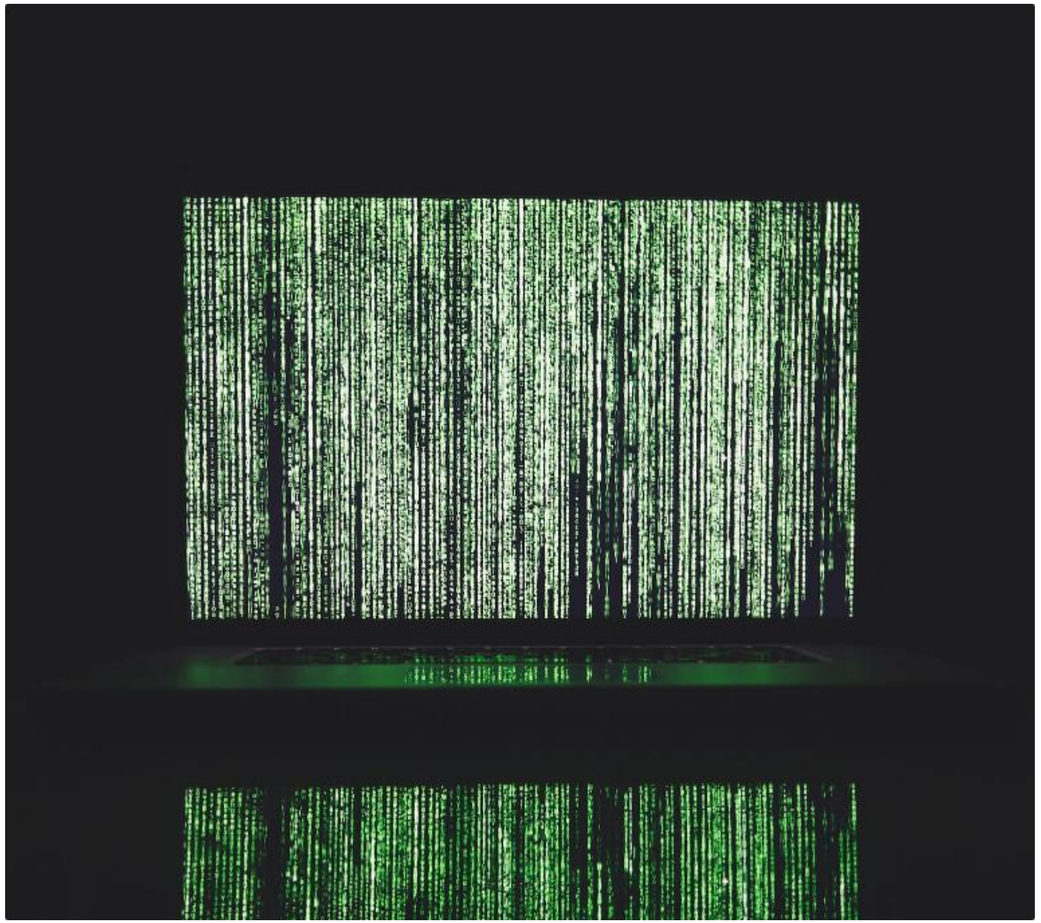
Establishing a bulletproof foundation through robust data management and experiment tracking.

Data Management

- Data Ingestion & ETL using **Apache Spark** or dbt.
- Data & Schema Versioning using **DVC** and cloud storage.
- Feature Stores like Feast or Tecton to eliminate training-serving skew.

Experiment Tracking

- Logging parameters and metrics via **MLflow** start_run().
- Saving artifacts and Git commit hashes for reproducibility.
- Run comparison using **Weights & Biases** or MLflow UI.



Stage 2: CI/CD & Automated Testing

The engine of the lifecycle, featuring automated quality gates that prevent model regression.

The CI Pipeline

- Code Quality: Standard unit tests using **pytest** and linters.
- Build & Train: Auto-training script using data from **DVC pull**.

The Model Quality Gate

- Performance: Evaluation on a blind test set for unbiased metrics.
- Regression Testing: Asserting new model is better than **production model**.
- Fairness: Validating data slices using tools like **Fairlearn**.

Stage 3: Registration & Deployment

Handover procedures from training to production using versioned registries and

Model Registry

- Lifecycle Management: Tags for **Staging**, **Production**, and Archived.
- Centralized Source using **MLflow** or cloud native registries.

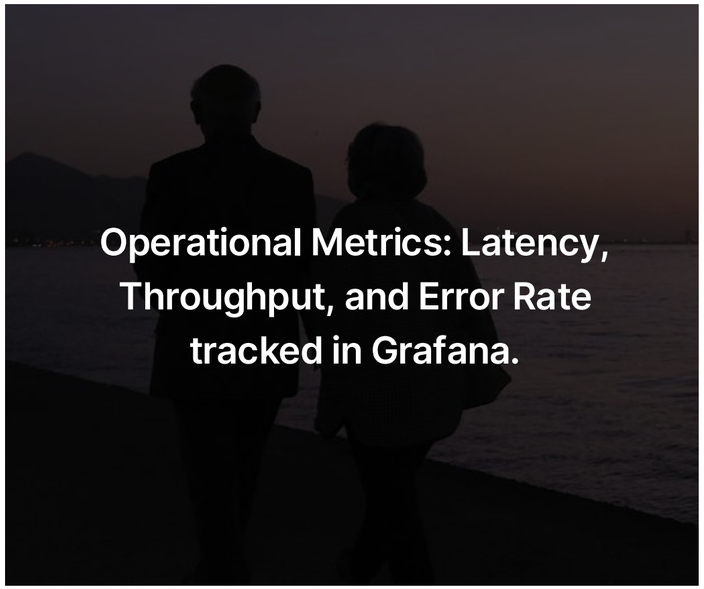
Deployment Strategies

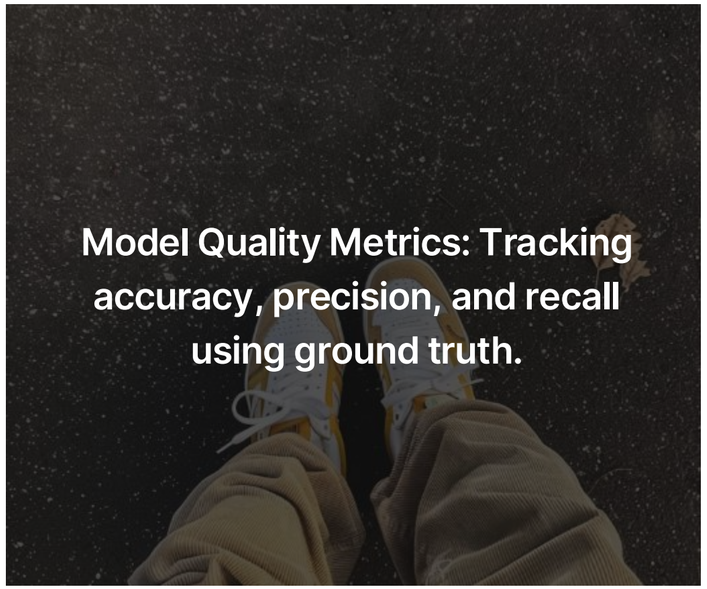
- Shadow Deployment: Running new model alongside old with zero risk.
- Canary Release: Gradual traffic increase to new model versions.
- Containerization: Packaging models and servers into **Docker** images.

Stage 4: Monitoring & Drift

Continuous production surveillance using operational metrics and statistical drift

1. Data drift (Covariate shift)
   When P(X) changes but P(Y|X) remains the same.
2. Label shift
   When P(Y) changes but P(X|Y) remains the same.
3. Concept drift
   When P(Y|X) changes but P(X) remains the same.





Stage 5: Automated Retraining

Completing the self-healing cycle through proactive triggers and orchestrated

1 Retraining Triggers

Retraining can be **Drift-Based** (proactive), **Performance-Based** (reactive), or **Scheduled** via cron.

2 Workflow Orchestration

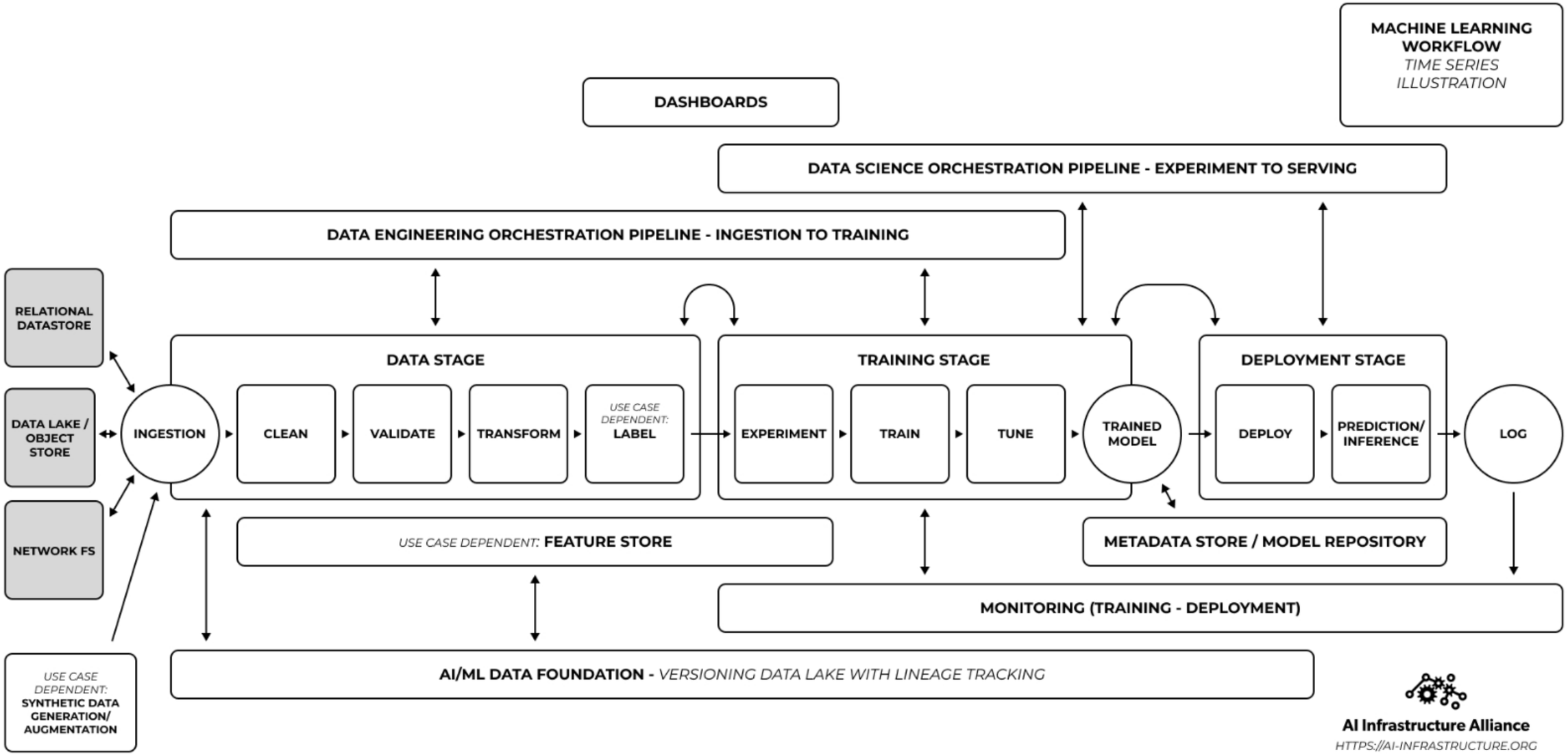
- Orchestrators like **Apache Airflow** or **Kubeflow** manage DAGs.
- Human-in-the-Loop: Ground truth labeling for low-confidence models.

Summary: A Complete Toolchain

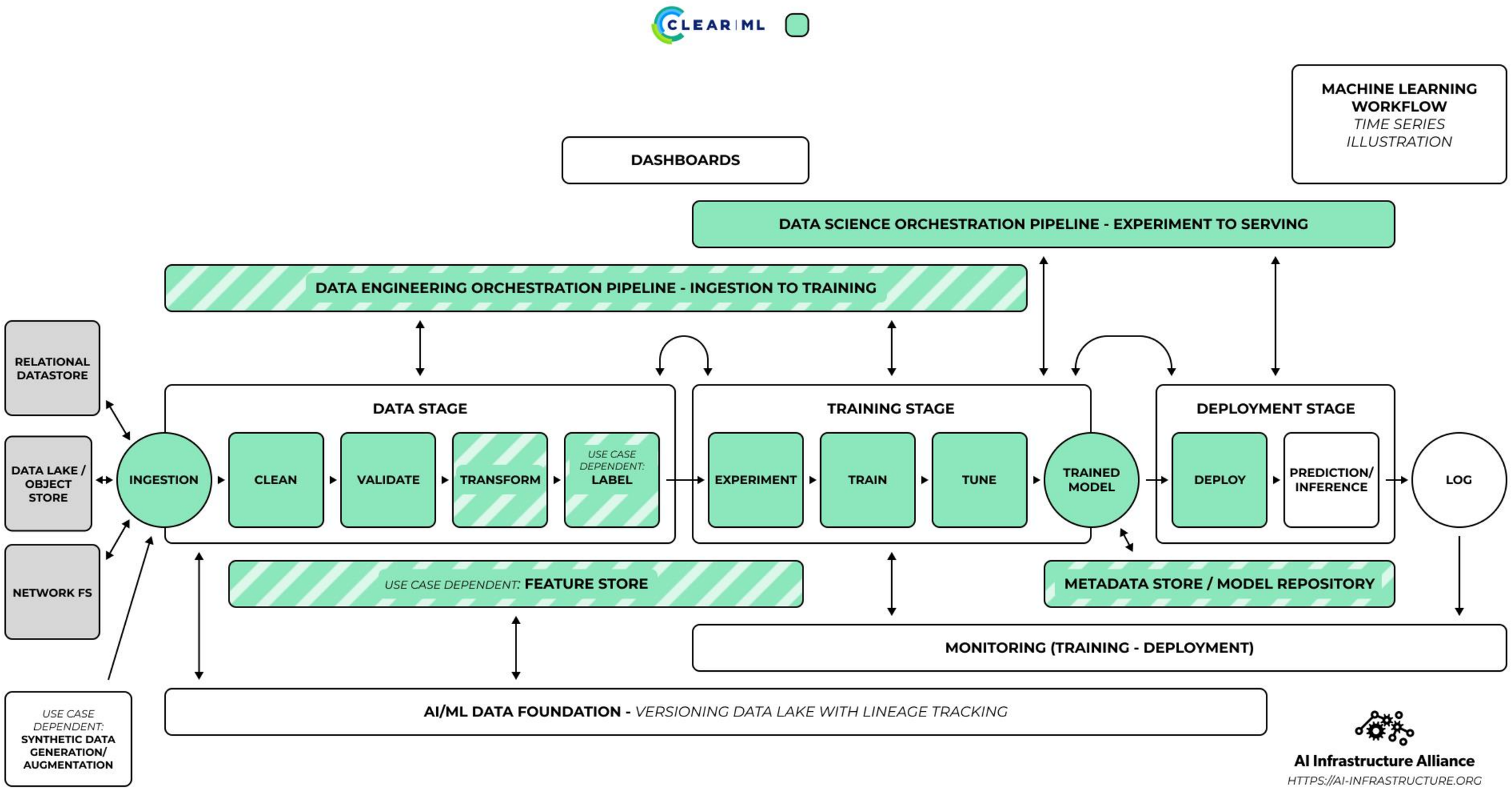
An integrated view of how open-source and managed tools combine to build a powerful MLOps platform.

Stage	Purpose	Example Toolchain
Code & Env	Versioning and dependencies	**Git (GitHub)**, **Docker**
Data & Features	Versioning and features	**DVC**, **S3/GCS**, **Feast**, **dbt**
Experiment Tracking	Logging model runs	**MLflow Tracking**
CI/CD Orchestration	Automating workflow	**GitHub Actions** or **Jenkins**
Model Testing	Quality gates and fairness	**Pytest**, **Fairlearn**
Model Registry	Versioning approved models	**MLflow Model Registry**
Model Serving	Scalable APIs	**Seldon Core**, **Kubernetes**, **SageMaker**
Monitoring	Dashboards and alerts	**Prometheus**, **Grafana**, **Evidently AI**
Retraining Trigger	Closing the training loop	**Apache Airflow** or **Kubeflow**

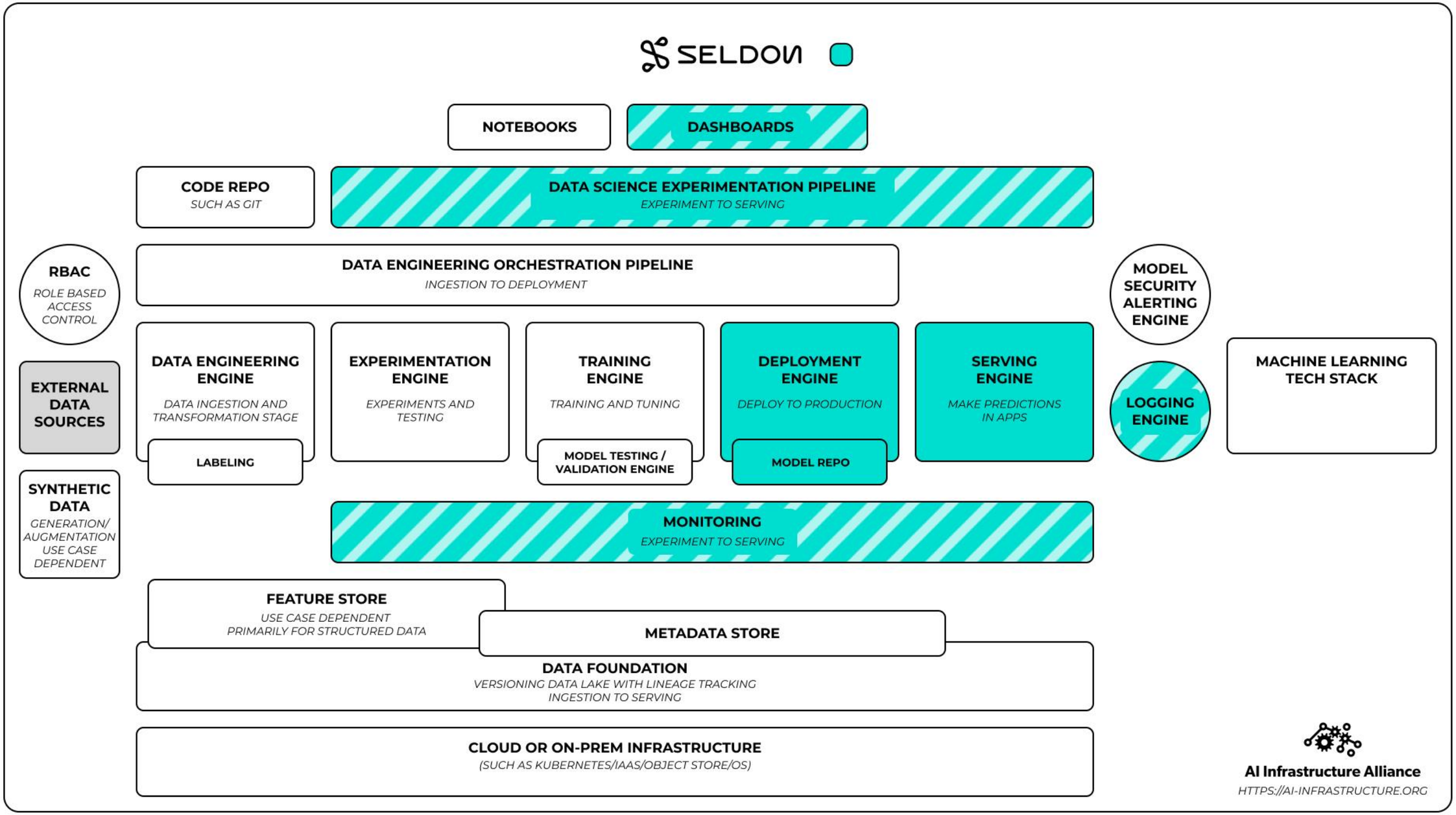
The Blueprints by AIIA



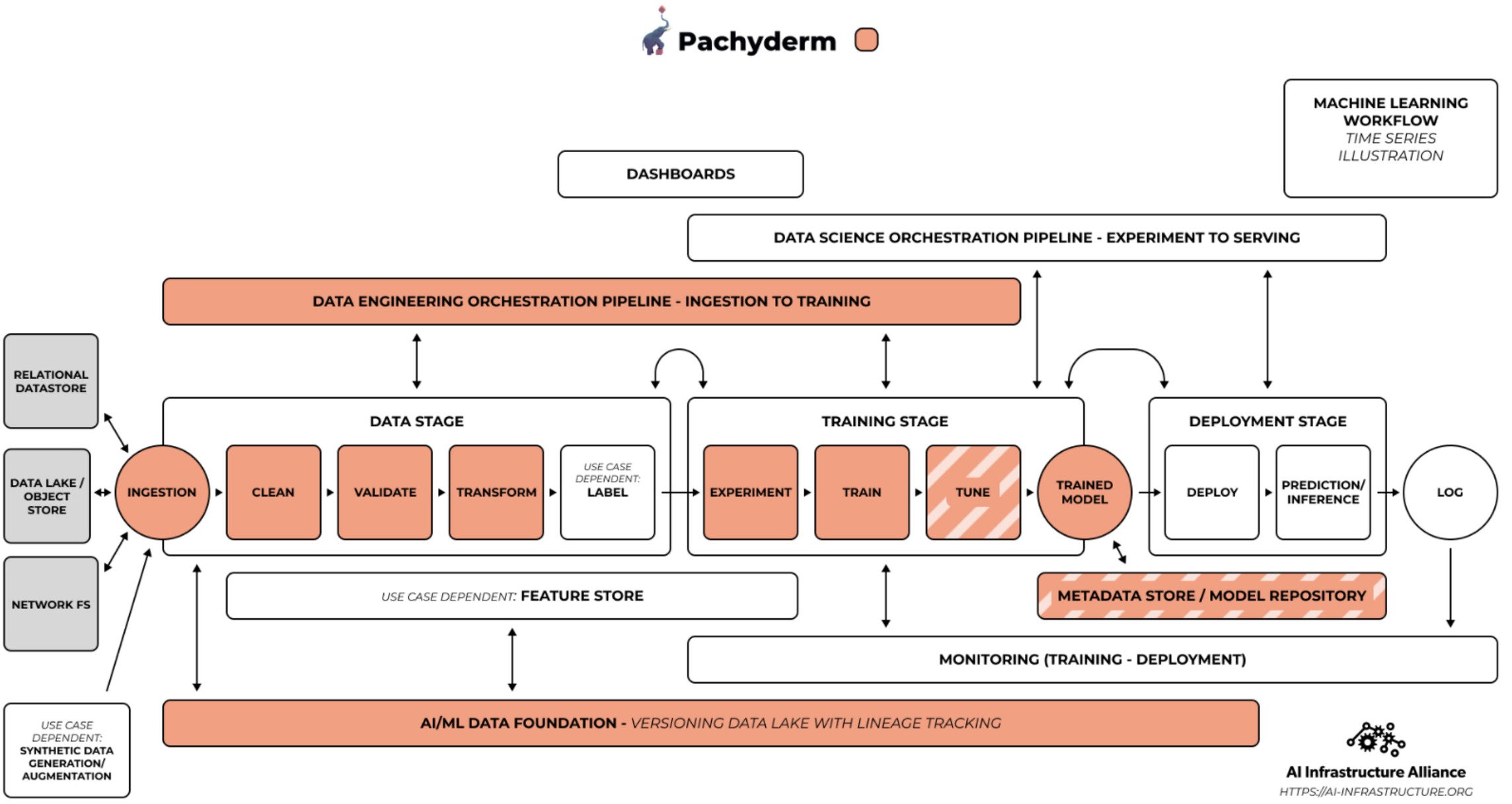
https://ai-infrastructure.org/why-we-started-the-aiia-and-what-it-means-for-the-rapid-evolution-of-the-canonical-stack-of-machine-learning/



https://ai-infrastructure.org/why-we-started-the-aiia-and-what-it-means-for-the-rapid-evolution-of-the-canonical-stack-of-machine-learning/

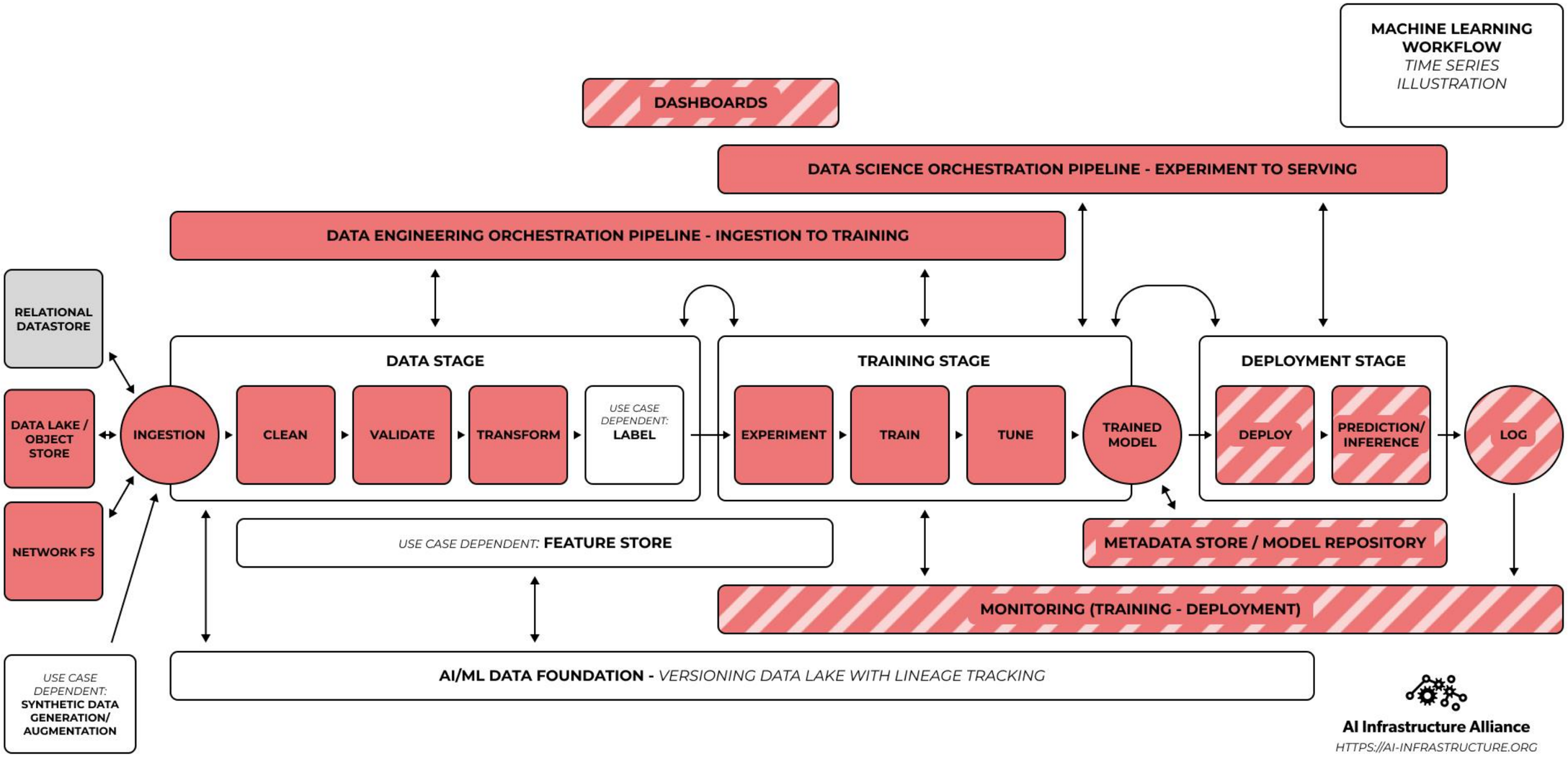


https://ai-infrastructure.org/why-we-started-the-aiia-and-what-it-means-for-the-rapid-evolution-of-the-canonical-stack-of-machine-learning/



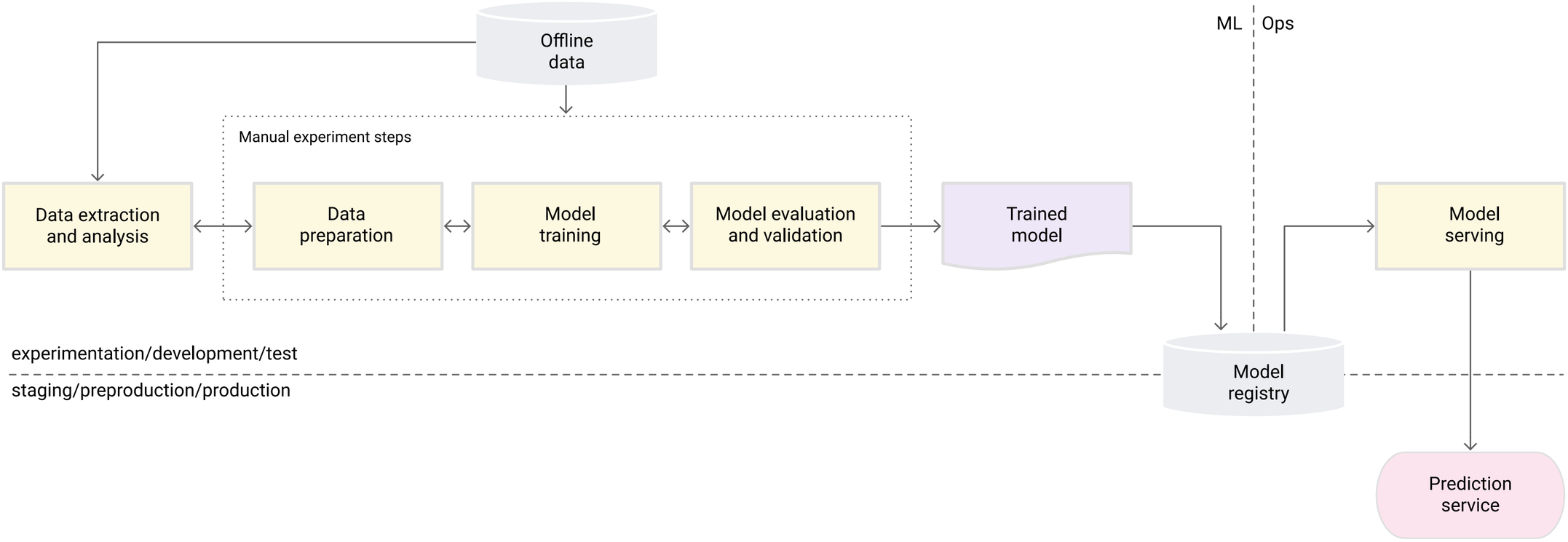
https://ai-infrastructure.org/why-we-started-the-aiia-and-what-it-means-for-the-rapid-evolution-of-the-canonical-stack-of-machine-learning/

neu.ro



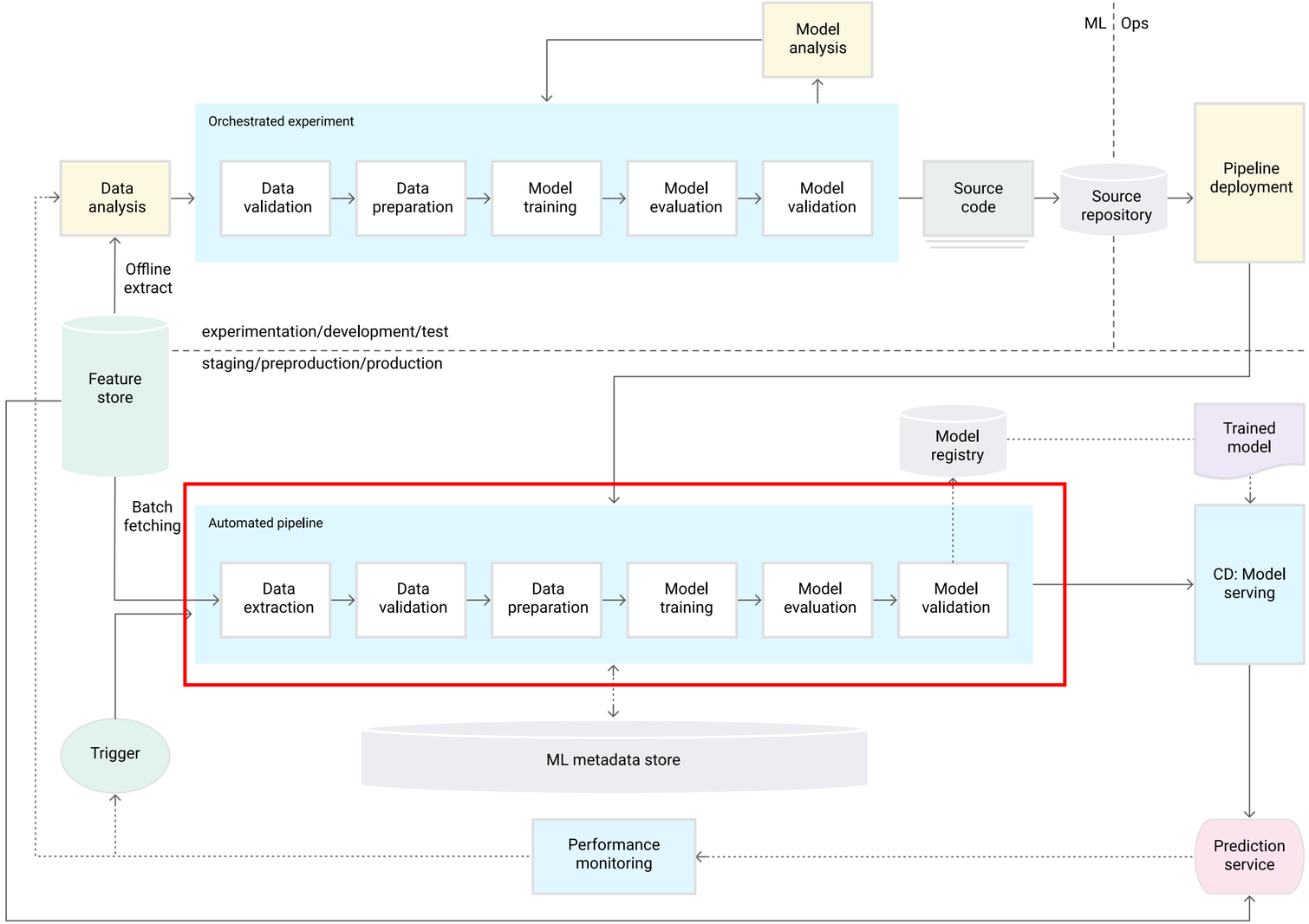
https://ai-infrastructure.org/why-we-started-the-aiia-and-what-it-means-for-the-rapid-evolution-of-the-canonical-stack-of-machine-learning/

MLOps level 0: Manual Process



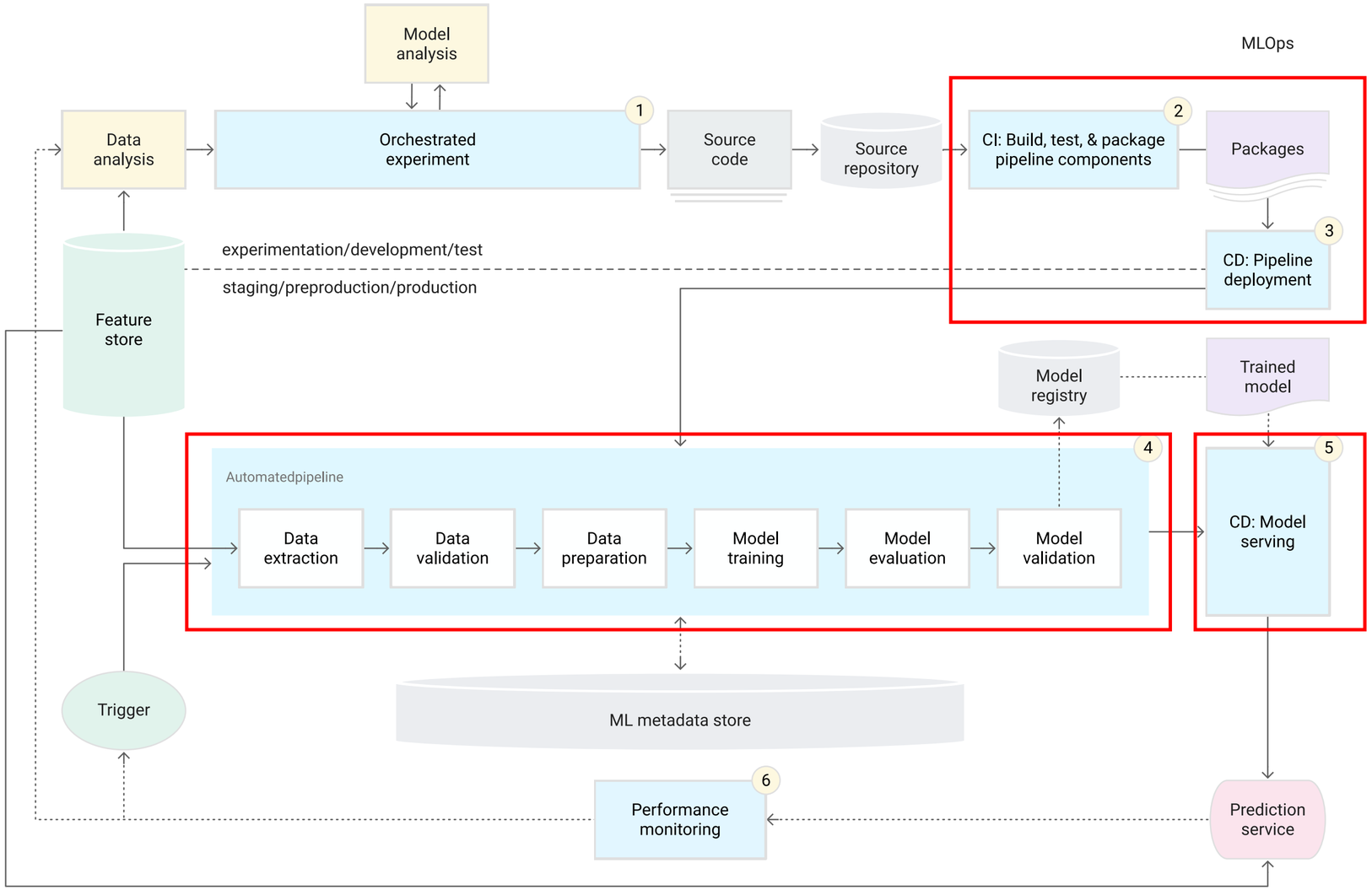
https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

MLOps level 1: ML pipeline automation



https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

MLOps level 2: CI/CD pipeline automation



https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning